

CS480P/580P Introduction to Natural Language Processing

Patrick H. Chen

September 16, 2025

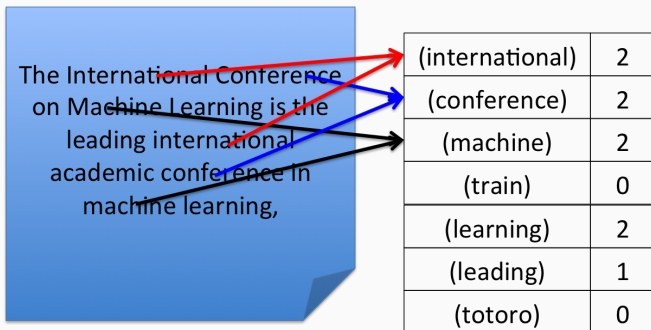
Lecture 8

Motivation

- Many unlabeled NLP data but very few labeled data
- Can we use large amount of unlabeled data to obtain meaningful representations of words/sentences?
- meaningful representations usually mean "embedding vectors"
- human-designed features vs learned features (key focus in early deep learning era)

Bag of Word

- A classical way to represent NLP data
- Each sentence (or document) is represented by a d -dimensional vector $\mathbf{x}$, where x_i is number of occurrences of word i .



number of features = number of potential words (very large)

Feature generation for documents

- Bag of n -gram features ($n = 2$):

The International Conference on Machine Learning is the leading international academic conference in machine learning,

(international)	2
(conference)	2
(machine)	2
(train)	0
(learning)	2
(leading)	1
(totoro)	0

(international conference)	1
(machine learning)	2
(leading international)	1
(totoro tiger)	0
(tiger woods)	0
(international academic)	1
(international academic)	1

10,000 words $\Rightarrow$ 10,000² potential features

Data Matrix (document)

- Use the bag-of-words matrix or the normalized version (TF-IDF) for a dataset (denoted by D):

$$\text{tfidf}(\text{doc}, \text{word}, D) = \text{tf}(\text{doc}, \text{word}) \cdot \text{idf}(\text{word}, D)$$

- $\text{tf}(\text{doc}, \text{word})$: term frequency

(word count in the document)/(total number of terms in the document)

- $\text{idf}(\text{word}, \text{Dataset})$: inverse document frequency

$\log((\text{Number of documents})/(\text{Number of documents with this word}))$

	angeles	los	new	post	times	york
d1	0	0	1	0	1	1
d2	0	0	1	1	0	1
d3	1	1	0	0	1	0

tf-idf

	angeles	los	new	post	times	york
d1	0	0	0.584	0	0.584	0.584
d2	0	0	0.584	1.584	0	0.584
d3	1.584	1.584	0	0	0.584	0

Bag of word + linear model

- Example: text classification (e.g., sentiment prediction, review score prediction)
- Linear model: $y \approx \text{sign}(\mathbf{w}^T \mathbf{x})$
(e.g., by linear SVM/logistic regression)
- w_i : the “contribution” of each word
- Not capturing the sequential information

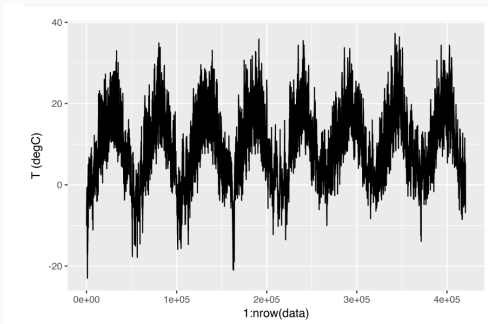
Time Series/Sequence Data

- Input: $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T\}$
 - Each $\mathbf{x}_t$ is the feature at time step t
 - Each $\mathbf{x}_t$ can be an d -dimensional vector
- Output: $\{y_1, y_2, \dots, y_T\}$
 - Each y_t is the output at step t
 - Multi-class output or Regression output:

$$y_t \in \{1, 2, \dots, L\} \quad \text{or} \quad y_t \in R$$

Example: Time Series Prediction

- Climate Data:
 - x_t : temperature at time t
 - y_t : temperature (or temperature change) at time $t + 1$
- Stock Price: Predicting stock price



Example: Language Modeling

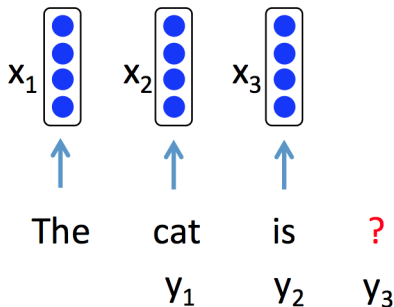
The cat is ?

Example: Language Modeling

The cat is ?

- x_t : one-hot encoding to represent the word at step t
([0, ..., 0, 1, 0, ..., 0])
- y_t : the next word

$y_t \in \{1, \dots, V\}$ V : Vocabulary size

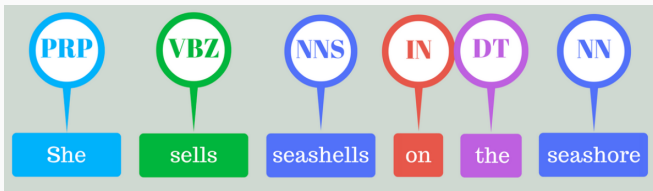


Example: POS Tagging

- Part of Speech Tagging:

Labeling words with their Part-Of-Speech (Noun, Verb, Adjective, ...)

- $\mathbf{x}_t$: a **vector** to represent the word at step t
- y_t : label of word t

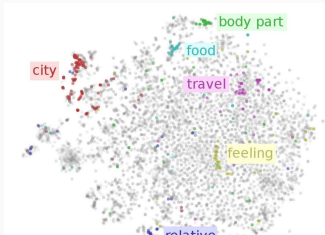


picture from <https://medium.com/analytics-vidhya/>

pos-tagging-using-conditional-random-fields-92077e5eaa31

Learning word embeddings

- Use large (unlabeled) corpus to learn a useful **word representation**
 - Learn a vector for each word based on the corpus
 - Hopefully the vector represents some semantic meaning
 - Can be used for many tasks
 - Replace the word embedding matrix for DNN models for classification/translation
- Two different perspectives but led to similar results:
 - Glove (Pennington et al., 2014)
 - Word2vec (Mikolov et al., 2013)



Context information

- Given a large text corpus, how to learn **low-dimensional features** to represent a word?
- For each word w_i , define the “**contexts**” of the word as the words surrounding it in an L -sized window:

$$w_{i-L-2}, w_{i-L-1}, \underbrace{w_{i-L}, \dots, w_{i-1}}_{\text{contexts of } w_i}, \underbrace{w_{i+1}, \dots, w_{i+L}}_{\text{contexts of } w_i}, w_{i+L+1}, \dots$$

- Get a collection of (word, context) pairs, denoted by D .

Examples

Source Text	Training Samples						
<table><tr><td>The</td><td>quick</td><td>brown</td></tr></table> fox jumps over the lazy dog. ➡	The	quick	brown	(the, quick) (the, brown)			
The	quick	brown					
<table><tr><td>The</td><td>quick</td><td>brown</td><td>fox</td></tr></table> jumps over the lazy dog. ➡	The	quick	brown	fox	(quick, the) (quick, brown) (quick, fox)		
The	quick	brown	fox				
<table><tr><td>The</td><td>quick</td><td>brown</td><td>fox</td><td>jumps</td></tr></table> over the lazy dog. ➡	The	quick	brown	fox	jumps	(brown, the) (brown, quick) (brown, fox) (brown, jumps)	
The	quick	brown	fox	jumps			
<table><tr><td>The</td><td>quick</td><td>brown</td><td>fox</td><td>jumps</td><td>over</td></tr></table> the lazy dog. ➡	The	quick	brown	fox	jumps	over	(fox, quick) (fox, brown) (fox, jumps) (fox, over)
The	quick	brown	fox	jumps	over		

(Figure from <http://mccormickml.com/2016/04/19/word2vec-tutorial-the-skip-gram-model/>)

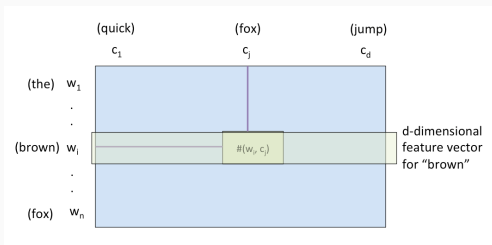
Use bag-of-words model

- Idea 1: Use the bag-of-words model to “describe” each word
- Assume we have context words $c_1, \dots, c_d$ in the corpus, compute

$\#(w, c_i) :=$ number of times the pair (w, c_i) appears in D

- For each word w , form a d -dimensional (sparse) vector to describe w

$$\#(w, c_1), \dots, \#(w, c_d),$$



PMI/PPMI Representation

- Similar to TF-IDF: Need to consider the frequency of each word and each context
- Instead of using co-occurrent count $\#(w, c)$, we can define pointwise mutual information:

$$\text{PMI}(w, c) = \log\left(\frac{\hat{P}(w, c)}{\hat{P}(w)\hat{P}(c)}\right) = \log \frac{\#(w, c)|D|}{\#(w)\#(c)},$$

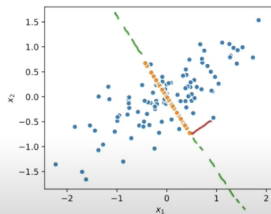
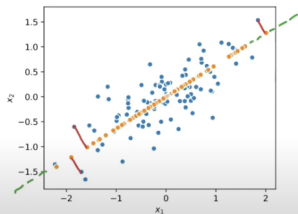
- $\#(w) = \sum_c \#(w, c)$: number of times word w occurred in D
- $\#(c) = \sum_w \#(w, c)$: number of times context c occurred in D , $|D|$: number of pairs in D
- Positive PMI (PPMI) usually achieves better performance:

$$\text{PPMI}(w, c) = \max(\text{PMI}(w, c), 0)$$

- M^{PPMI} : an n by d word feature matrix, each row is a word and each column is a context

PCA Intuition (<https://www.kamperh.com/data414/>)

View 1: Maximising variance



Principal Component Analysis

1. Dimension Reduction Technique
2. Find the projection where the variance is maximally preserved.
3. $\arg \max_{w^T w=1} w^T X^T X w$

PCA by SVD (<https://www.kamperh.com/data414/>)

PCA: Relationship to SVD

Singular value decomposition:

$$\begin{matrix} N \times D & N \times N & D \times D \\ \underline{X} = \underline{U} \underline{S} \underline{V}^T \\ & N \times D \end{matrix}$$

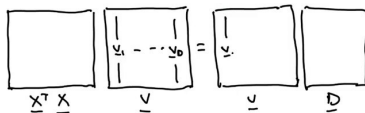
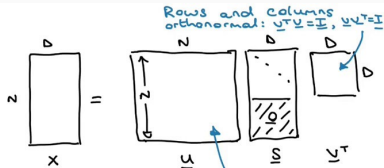
Relationship to PCA:

Take SVD of the design matrix $\underline{X}$:

$$\underline{X} = \underline{U} \underline{S} \underline{V}^T$$

$$\begin{aligned} \text{Then } \underline{X}^T \underline{X} &= \underline{V} \underline{S}^T \underline{U}^T \underline{U} \underline{S} \underline{V}^T \\ &= \underline{V} \underline{S}^T \underline{S} \underline{V}^T = \underline{V} \underline{D} \underline{V}^T \end{aligned}$$

$$(\underline{X}^T \underline{X}) \underline{V} = \underline{V} \underline{D}$$



Low-dimensional embedding

- Perform PCA/SVD on the sparse feature matrix:

$$M^{\text{PPMI}} \approx U_k \Sigma_k V_k^T$$

Then $W^{\text{SVD}} = U_k \Sigma_k$ is the context representation of each word

(Each row is a k -dimensional feature for a word)

Generalized Low-rank Embedding

- SVD basis will minimize

$$\min_{W,V} \|M^{\text{PPMI}} - WV^T\|_F^2$$

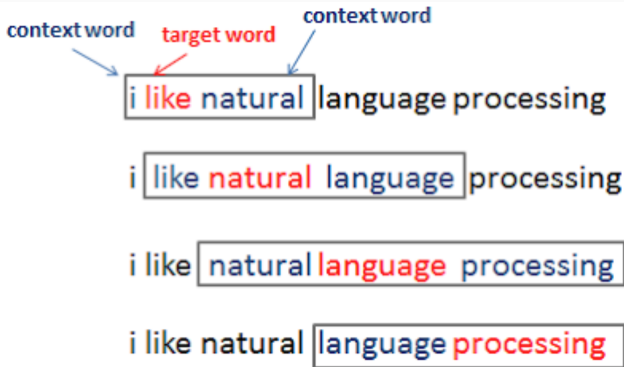
- Glove (Pennington et al., 2014)
 - Negative sampling (less weights to 0s in M^{PPMI})
 - Adding bias term:

$$M^{\text{PPMI}} \approx WV^T + \mathbf{b}_w \mathbf{e}^T + \mathbf{e} \mathbf{b}_c^T$$

- Use W or V as the word embedding matrix

Word2vec (Mikolov et al., 2013)

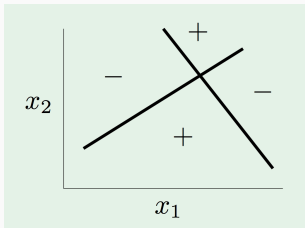
- A neural network model for learning word embeddings
- Main idea:
 - Predict the target words based on the neighbors (CBOW)
 - Predict neighbors given the target words (Skip-gram)



Neural Networks

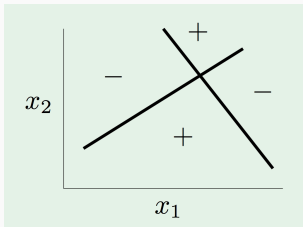
Another way to introduce nonlinearity

- How to generate this nonlinear hypothesis?

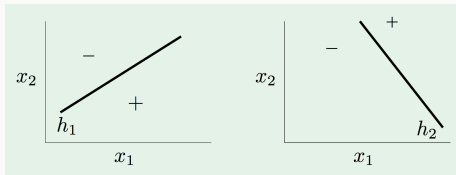


Another way to introduce nonlinearity

- How to generate this nonlinear hypothesis?



- Combining multiple linear hyperplanes to construct nonlinear hypothesis

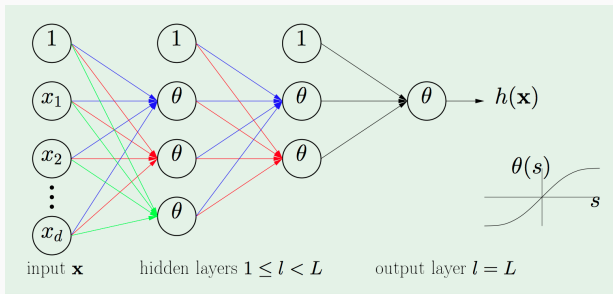


Neural Network

- Input layer: d neurons (input features)
- Neurons from layer 1 to L : Linear combination of previous layers + activation function

$$\theta(\mathbf{w}^T \mathbf{x}), \quad \theta : \text{activation function}$$

- Final layer: one neuron $\Rightarrow$ prediction by $\text{sign}(h(\mathbf{x}))$

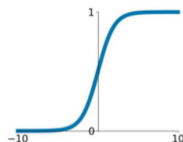


(figure from "learning from data")

Activation Function

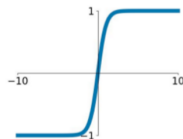
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



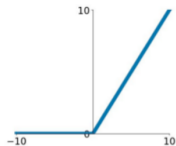
tanh

$$\tanh(x)$$



ReLU

$$\max(0, x)$$



Formal Definitions

$$\text{Weight: } w_{ij}^{(l)} \quad \left\{ \begin{array}{ll} 1 \leq l \leq L & : \text{layers} \\ 0 \leq i \leq d^{(l-1)} & : \text{inputs} \\ 1 \leq j \leq d^{(l)} & : \text{outputs} \end{array} \right.$$

bias: $b_j^{(l)}$: added to the j -th neuron in the l -th layer

Formal Definitions

$$\text{Weight: } w_{ij}^{(l)} \quad \left\{ \begin{array}{ll} 1 \leq l \leq L & : \text{layers} \\ 0 \leq i \leq d^{(l-1)} & : \text{inputs} \\ 1 \leq j \leq d^{(l)} & : \text{outputs} \end{array} \right.$$

bias: $b_j^{(l)}$: added to the j -th neuron in the l -th layer

j -th neuron in the l -th layer:

$$x_j^{(l)} = \theta(s_j^{(l)}) = \theta\left(\sum_{i=0}^{d^{(l-1)}} w_{ij}^{(l)} x_i^{(l-1)} + b_j^{(l)}\right)$$

Formal Definitions

$$\text{Weight: } w_{ij}^{(l)} \quad \begin{cases} 1 \leq l \leq L & : \text{layers} \\ 0 \leq i \leq d^{(l-1)} & : \text{inputs} \\ 1 \leq j \leq d^{(l)} & : \text{outputs} \end{cases}$$

bias: $b_j^{(l)}$: added to the j -th neuron in the l -th layer

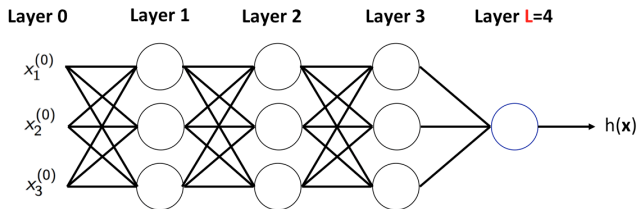
j -th neuron in the l -th layer:

$$x_j^{(l)} = \theta(s_j^{(l)}) = \theta\left(\sum_{i=0}^{d^{(l-1)}} w_{ij}^{(l)} x_i^{(l-1)} + b_j^{(l)}\right)$$

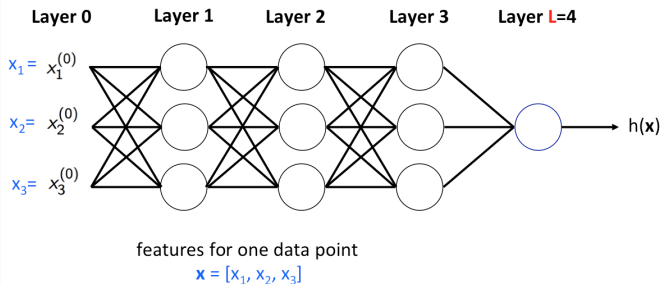
Output:

$$h(\mathbf{x}) = x_1^{(L)}$$

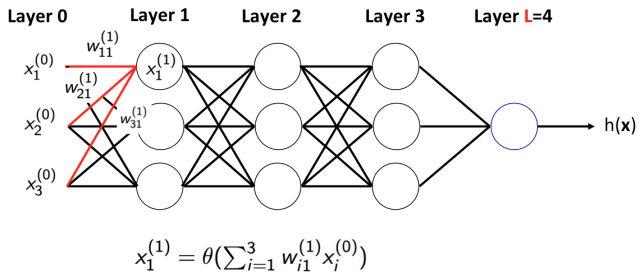
Forward propagation



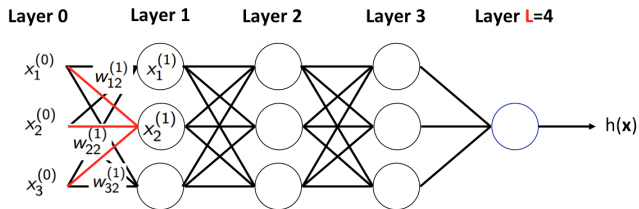
Forward propagation



Forward propagation

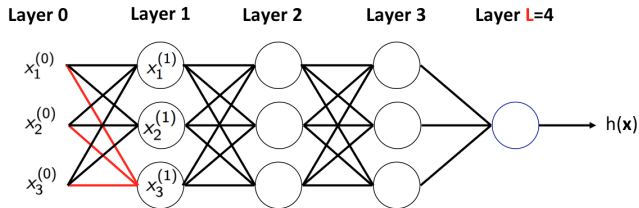


Forward propagation

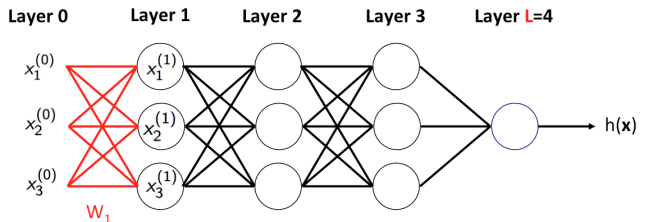


$$x_2^{(1)} = \theta(\sum_{i=1}^3 w_{i2}^{(1)} x_i^{(0)})$$

Forward propagation

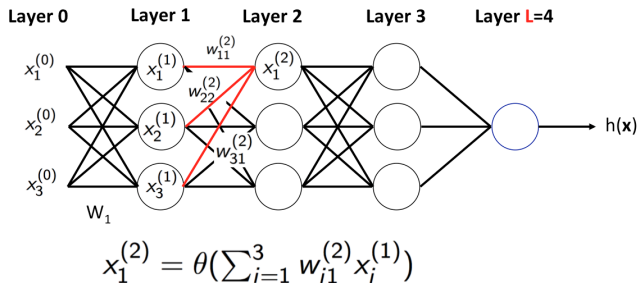


Forward propagation

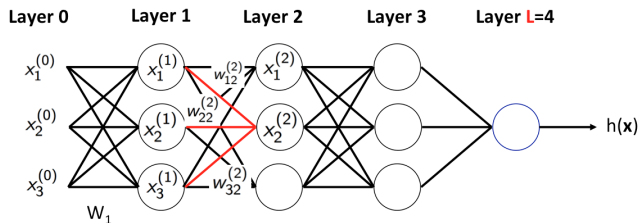


$$\mathbf{x}^{(1)} = \begin{bmatrix} x_1^{(1)} \\ x_2^{(1)} \\ x_3^{(1)} \end{bmatrix} = \theta \left(\begin{bmatrix} w_{11}^{(1)} & w_{21}^{(1)} & w_{31}^{(1)} \\ w_{12}^{(1)} & w_{22}^{(1)} & w_{32}^{(1)} \\ w_{13}^{(1)} & w_{23}^{(1)} & w_{33}^{(1)} \end{bmatrix} \times \begin{bmatrix} x_1^{(0)} \\ x_2^{(0)} \\ x_3^{(0)} \end{bmatrix} \right) = \theta(\mathbf{W}_1 \mathbf{x}^{(0)})$$

Forward propagation

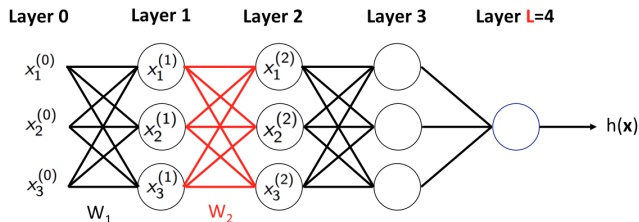


Forward propagation



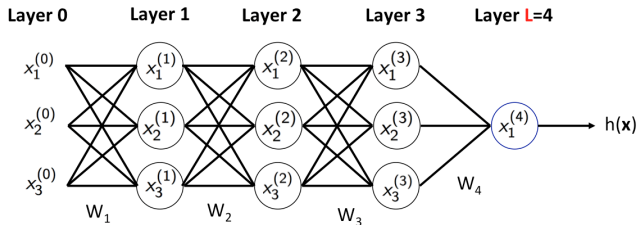
$$x_2^{(2)} = \theta(\sum_{i=1}^3 w_{i2}^{(2)} x_i^{(1)})$$

Forward propagation



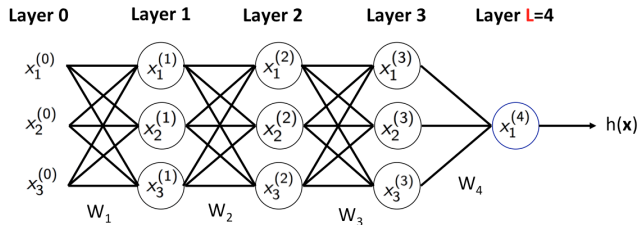
$$\mathbf{x}^{(2)} = \begin{bmatrix} x_1^{(2)} \\ x_2^{(2)} \\ x_3^{(2)} \end{bmatrix} = \theta \left(\begin{bmatrix} w_{11}^{(2)} & w_{21}^{(2)} & w_{31}^{(2)} \\ w_{12}^{(2)} & w_{22}^{(2)} & w_{32}^{(2)} \\ w_{13}^{(2)} & w_{23}^{(2)} & w_{33}^{(2)} \end{bmatrix} \times \begin{bmatrix} x_1^{(1)} \\ x_2^{(1)} \\ x_3^{(1)} \end{bmatrix} \right) = \theta(W_2 \mathbf{x}^{(1)})$$

Forward propagation



$$\begin{aligned} h(\mathbf{x}) &= x_1^{(4)} = \theta(W_4 \mathbf{x}^{(3)}) = \theta(W_4 \theta(W_3 \mathbf{x}^{(2)})) \\ &= \dots = \theta(W_4 \theta(W_3 \theta(W_2 \theta(W_1 \mathbf{x})))) \end{aligned}$$

Forward propagation



$$\begin{aligned} h(\mathbf{x}) &= x_1^{(4)} = \theta(W_4 \mathbf{x}^{(3)}) = \theta(W_4 \theta(W_3 \mathbf{x}^{(2)})) \\ &= \dots = \theta(W_4 \theta(W_3 \theta(W_2 \theta(W_1 \mathbf{x})))) \end{aligned}$$

With the bias term:

$$h(\mathbf{x}) = \theta(W_4 \theta(W_3 \theta(W_2 \theta(W_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) + \mathbf{b}_3) + \mathbf{b}_4)$$

Training

- Weights $W = \{W_1, \dots, W_L\}$ and bias $\{\mathbf{b}_1, \dots, \mathbf{b}_L\}$ determine $h(\mathbf{x})$
- Learning the weights: solve ERM with SGD.
- Loss on example $(\mathbf{x}_n, y_n)$ is

$$e(h(\mathbf{x}_n), y_n) = e(W)$$

Training

- Weights $W = \{W_1, \dots, W_L\}$ and bias $\{\mathbf{b}_1, \dots, \mathbf{b}_L\}$ determine $h(\mathbf{x})$
- Learning the weights: solve ERM with SGD.
- Loss on example $(\mathbf{x}_n, y_n)$ is

$$e(h(\mathbf{x}_n), y_n) = e(W)$$

- To implement SGD, we need the gradient

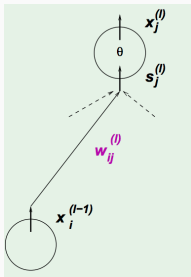
$$\nabla e(W) : \left\{ \frac{\partial e(W)}{\partial w_{ij}^{(l)}} \right\} \text{ for all } i, j, l$$

(for simplicity we ignore bias in the derivations)

Computing Gradient $\frac{\partial e(W)}{\partial w_{ij}^{(l)}}$

- Use **chain rule**:

$$\frac{\partial e(W)}{\partial w_{ij}^{(l)}} = \frac{\partial e(W)}{\partial s_j^{(l)}} \times \frac{\partial s_j^{(l)}}{\partial w_{ij}^{(l)}}$$



$$s_j^{(l)} = \sum_{i=1}^d x_i^{(l-1)} w_{ij}^{(l)}$$

- We have $\frac{\partial s_j^{(l)}}{\partial w_{ij}^{(l)}} = x_i^{(l-1)}$

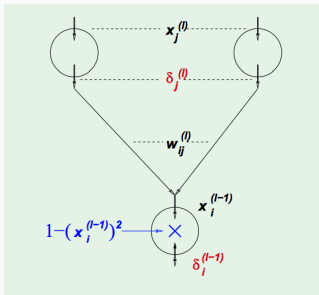
Computing Gradient $\frac{\partial e(W)}{\partial w_{ij}^{(l)}}$ (Skip)

- Define $\delta_j^{(l)} := \frac{\partial e(W)}{\partial s_j^{(l)}}$
- Compute by **layer-by-layer**:

$$\begin{aligned}\delta_i^{(l-1)} &= \frac{\partial e(W)}{\partial s_i^{(l-1)}} \\&= \sum_{j=1}^d \frac{\partial e(W)}{\partial s_j^{(l)}} \times \frac{\partial s_j^{(l)}}{\partial x_i^{(l-1)}} \times \frac{\partial x_i^{(l-1)}}{\partial s_i^{(l-1)}} \\&= \sum_{j=1}^d \delta_j^{(l)} \times w_{ij}^{(l)} \times \theta'(s_i^{(l-1)}),\end{aligned}$$

where $\theta'(s) = 1 - \theta^2(s)$ for $\tanh$

- $\delta_i^{(l-1)} = (1 - (x_i^{(l-1)})^2) \sum_{j=1}^d w_{ij}^{(l)} \delta_j^{(l)}$



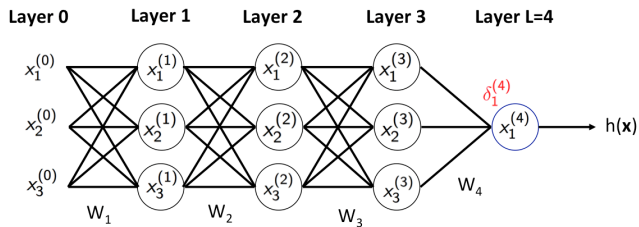
Final layer

(Assume square loss)

- $e(W) = (x_1^{(L)} - y_n)^2$
 $x_1^{(L)} = \theta(s_1^{(L)})$
- So,

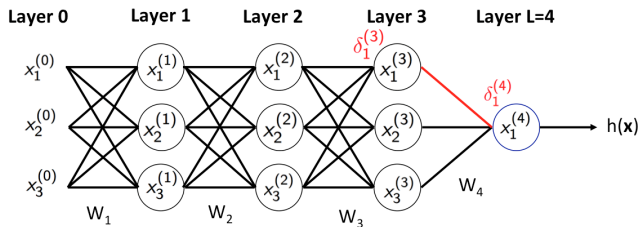
$$\begin{aligned}\delta_1^{(L)} &= \frac{\partial e(W)}{\partial s_1^{(L)}} \\ &= \frac{\partial e(W)}{\partial x_1^{(L)}} \times \frac{\partial x_1^{(L)}}{\partial s_1^{(L)}} \\ &= 2(x_1^{(L)} - y_n) \times \theta'(s_1^{(L)})\end{aligned}$$

Backward propagation



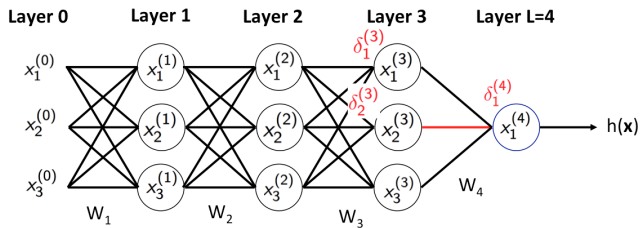
$$\delta_1^{(4)} = 2(x_1^{(4)} - y_n) \times (1 - (x_1^{(4)})^2)$$

Backward propagation



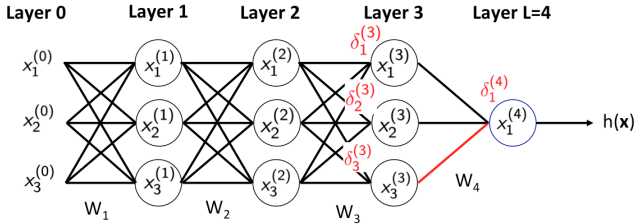
$$\delta_1^{(3)} = (1 - (x_1^{(3)})^2) \times \delta_1^{(4)} \times w_{11}^{(4)}$$

Backward propagation



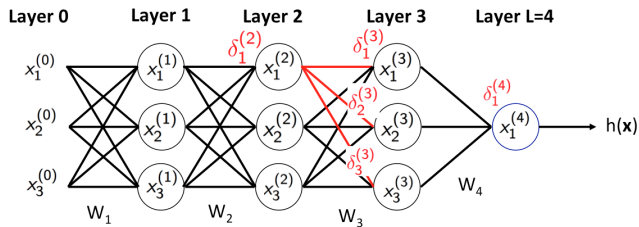
$$\delta_2^{(3)} = (1 - (x_2^{(3)})^2) \times \delta_1^{(4)} \times w_{21}^{(4)}$$

Backward propagation



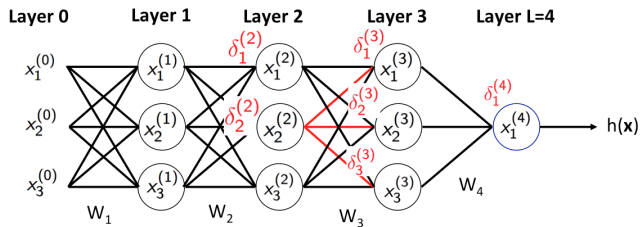
$$\delta_3^{(3)} = (1 - (x_3^{(3)})^2) \times \delta_1^{(4)} \times w_{31}^{(4)}$$

Backward propagation



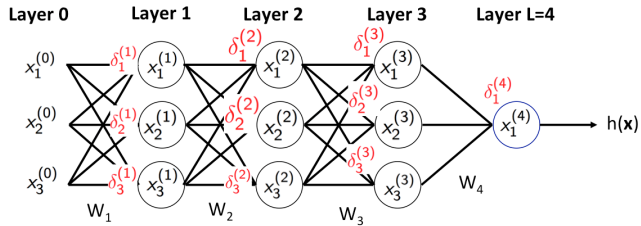
$$\delta_1^{(2)} = (1 - (x_1^{(2)})^2) \sum_{j=1}^3 \delta_j^{(3)} w_{1j}^{(3)}$$

Backward propagation



$$\delta_2^{(2)} = (1 - (x_2^{(2)})^2) \sum_{j=1}^3 \delta_j^{(3)} w_{2j}^{(3)}$$

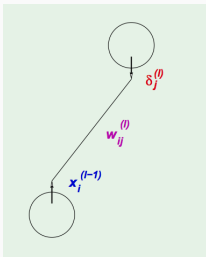
Backward propagation



Stochastic Gradient Descent (Continue Here.

SGD for neural networks

- Initialize all weights $w_{ij}^{(l)}$ **at random**
- For $\text{iter} = 0, 1, 2, \dots$
 - Sample a data point
 - **Forward**: Compute all $x_j^{(l)}$ from input to output
 - **Backward**: Compute all $\delta_j^{(l)}$ from output to input
 - Update all the weights $w_{ij}^l \leftarrow w_{ij}^{(l)} - \eta x_i^{(l-1)} \delta_j^{(l)}$



Min-batch SGD

- Initialize all weights $w_{ij}^{(l)}$ **at random**
- For $\text{iter} = 0, 1, 2, \dots$
 - Sample a batch of $|B|$ data points
 - **Forward**: Compute all $x_j^{(l)}$ from input to output for all samples
(parallel matrix/tensor products)
 - **Backward**: Compute all $\delta_j^{(l)}$ from output to input
(parallel matrix/tensor products)
 - Aggregate to get the average gradient
 - Update the weights

Backpropagation

- Just an automatic way to apply **chain rule** to compute gradient
- Auto-differentiation (AD) — as long as we define derivative for each **basic function**, we can use AD to compute any of their compositions
- Implemented in most deep learning packages
(e.g., pytorch, tensorflow)

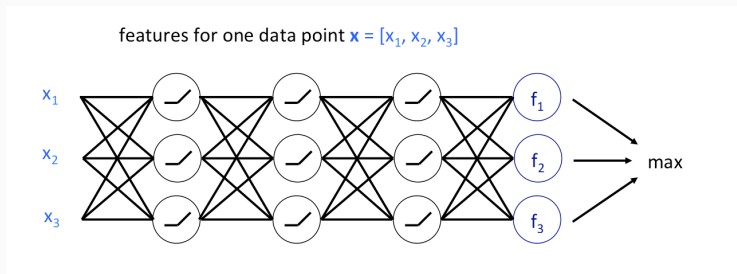
Backpropagation

- Just an automatic way to apply **chain rule** to compute gradient
- Auto-differentiation (AD) — as long as we define derivative for each **basic function**, we can use AD to compute any of their compositions
- Implemented in most deep learning packages
(e.g., pytorch, tensorflow)
- Auto-differentiation needs to store all the intermediate nodes of each sample
 - ⇒ Memory cost $\geq$ number of neurons $\times$ batch size
 - ⇒ This poses a constraint on the batch size

Multiclass Classification

- K classes: K neurons in the final layer
- Output of each f_i is the score of class i

Taking $\arg \max_i f_i(x)$ as the prediction



Multiclass loss

- Softmax function: transform output to probability:

$$[f_1, \dots, f_K] \rightarrow [p_1, \dots, p_K]$$

where $p_i = \frac{e^{f_i}}{\sum_{j=1}^K e^{f_j}}$

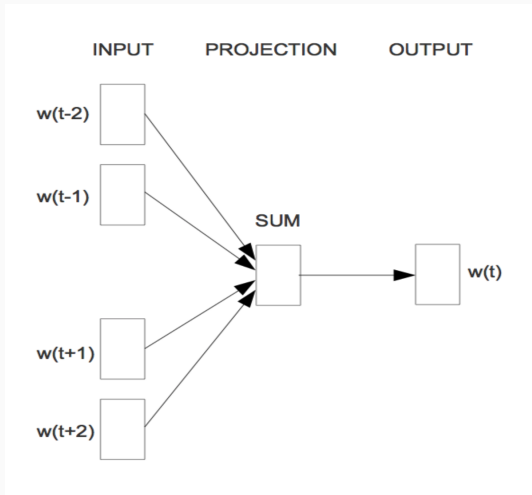
- Cross-entropy loss:

$$L = - \sum_{i=1}^K y_i \log(p_i)$$

where y_i is the i -th label

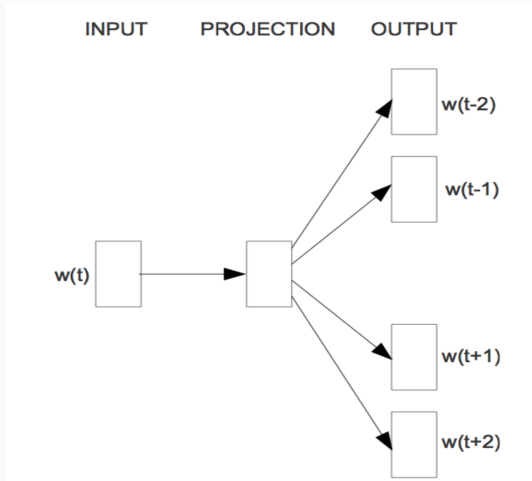
CBOW (Continuous Bag-of-Word model)

- Predict the target words based on the neighbors



Skip-gram

- Predict neighbors using target word



More on skip-gram

- Learn the probability $P(w_{t+j}|w_t)$: the probability to see w_{t+j} in target word w_t 's neighborhood
- Every word has two embeddings:
 - v_i serves as the role of target
 - u_i serves as the role of context
- Model probability as softmax:

$$P(o|c) = \frac{e^{u_o^T v_c}}{\sum_{w=1}^W e^{u_w^T v_c}}$$

- Learn embeddings to minimize the cross entropy loss

$$\min_{\{u_i\}, \{v_j\}} \sum_{(p,q) \in \text{positive pairs}} -\log\left(\frac{e^{u_p^T v_q}}{\sum_{r \neq p} e^{u_r^T v_q}}\right)$$

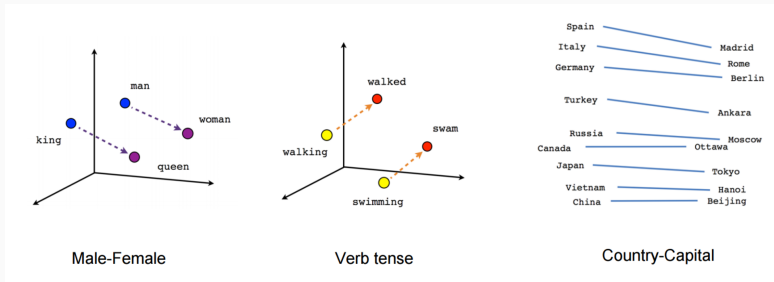
- Considering all $r \neq p$ is time consuming (size of vocabulary is large)
- Negative sampling:

$$\min_{\{u_i\}, \{v_j\}} \sum_{(p,q) \in \text{positive pairs}} -\log\left(\frac{e^{u_p^T v_q}}{\sum_{r \in D_{\text{negative}}} e^{u_r^T v_q}}\right)$$

where D_{negative} contains a subset of negative samples.

Results

The low-dimensional embeddings are (often) meaningful:



(Figure from
<https://www.tensorflow.org/tutorials/word2vec>)